

Project Report: Migrating from Vanilla JavaScript to React

Introduction: The Shift in Mindset

The primary motivation for refactoring this application was to transition from an imperative programming style to a declarative one. In my previous Vanilla JS version, I was manually responsible for DOM manipulation—step-by-step telling the browser how to update elements (e.g., `document.getElementById(...).style.display = 'block'`).

Adopting **React** allowed me to define *what* the UI should look like based on the current data state. React handles the heavy lifting of updating the DOM to match that state, making the code more predictable and easier to debug.

1. The Environment Setup

To support this modern architecture, I moved away from simple HTML/JS files and established a build environment, following the guides on MDN Web Docs:

- **Node.js & NPM:** I installed Node.js to act as the runtime engine for my build tools. I utilized npm (Node Package Manager) to handle dependencies, allowing me to integrate robust libraries rather than writing core framework logic from scratch.
- **Vite:** I used Vite as my build tool. It acts as the "translator," converting modern React code (JSX) into the plain HTML/JS the browser understands.
- **Key Configuration Files:**
 - **package.json:** This is the recipe for my project. It lists metadata and dependencies (like React).
 - **package-lock.json:** This locks the specific versions of every package to ensure the app runs the same on my machine as it does on yours.
 - **main.jsx:** The entry point where the React app attaches to the index.html file.

2. Component-Based Architecture

The most significant change was thinking in Components. Instead of writing one massive HTML file, I broke the interface into reusable, independent pieces. I learned the fundamentals of this approach through the React.dev "Your First Component" guide.

- **Concept:** A component is simply a JavaScript function that returns UI (JSX).
- **Implementation:** For example, in my `Adventures.jsx`, I reused card layouts for different lessons. In `Bots.jsx`, I encapsulated video and text overlays into sections.
- **Composability:** I learned that components can hold other components. My `App.jsx` acts as a container that holds the Navbar, the current Page, and the Footer.

3. Application Architecture: The App.jsx Hub

I configured `App.jsx` to be the "Traffic Controller" of the application.

- **Routing:** I integrated `react-router-dom` to turn the project into a Single Page Application (SPA). This allows users to transition between pages without a hard refresh. I followed

the **React Router Official Tutorial** to set up the `<BrowserRouter>` and `<Routes>` structure.

- The Structure:

Inside `App.jsx`, I defined my routes. The Navbar and Footer sit outside the `<Routes>` block, ensuring they remain persistent while the content between them swaps out based on the URL.

4. Mastering React Hooks

Standard JavaScript functions are stateless—they don't "remember" anything. To give my functional components memory and timing capabilities, I learned to use Hooks.

- `useRef` (The Reference Hook):

In Vanilla JS, I would select elements using `document.querySelector`. In React, I learned to use `useRef` to directly access elements. For example, in `Bots.jsx`, I used `heroVideoRef` to control video playback without querying the DOM globally.

- `useEffect` (The Timing/Sync Hook):

This was crucial for handling "side effects"—things that happen outside the UI rendering.

- *Usage:* In `LessonPlayer.jsx`, I used `useEffect` to save the user's progress to `localStorage` every time the `slideId` changed.
- *Usage:* I also used it to create a custom hook, `useVideoAutoplay`, which sets up an `IntersectionObserver` to automatically play/pause videos when they scroll into view.

- `useParams` (Dynamic Data Hook):

This is perhaps the most powerful feature I implemented. In `LessonPlayer.jsx`, I do not hardcode lessons. Instead, I use `useParams()` to grab the `courseId`, `lessonId`, and `slideId` directly from the browser URL.

- *Result:* One single component (`LessonPlayer`) can render infinite unique lessons by simply reading the URL parameters.

5. Key Architectural Improvements (The "Why")

- From Hardcoded Files to Dynamic Templates

- ◆ The Old Way: I had separate HTML files for different lessons (`adventure_details4.html`, `About-mbot4.html`) and separate scripts (`script.js`, `script_city.js`). This meant fixing a bug in one lesson required copying that fix to every other file.
- ◆ The React Way: I created a single **template component** (`LessonPlayer.jsx`). It adapts its content based on the URL parameters (`:lessonId`). This drastically reduces code duplication and makes maintenance easier.

- From Multi-Page to Single-Page Application (SPA)

- The Old Way: Navigation relied on standard `<a href>` links, causing the

- browser to reload the entire page and redownload assets on every click.
- **The React Way:** Using `react-router-dom`, the app swaps components instantly without refreshing the browser. This creates a smoother, "app-like" experience.
-

6. Adapting Data for the React Ecosystem

In my previous Vanilla JS version, I had already established a "content file" to avoid hardcoding text into HTML. However, moving to React required me to refactor this existing data structure to support modern module bundling and state management.

- **From Global Variables to Modules:** Originally, my data was a simple global variable accessible to any script on the page. I updated this to use ES6 Modules (specifically adding `export default`), which allows me to strictly control where this data is imported. This prevents "namespace pollution" and ensures that only the `LessonPlayer` component accesses the course material.
Component-Ready Structure: I reorganized the internal data objects to match the "Props" expected by my React components. For example, I grouped all quiz-related questions and options into a nested `quizDetails` object. This allows me to pass a single object to my `<Quiz />` component, keeping the parent logic clean.
- **Routing Alignment:** I verified that my data array aligned perfectly with the `useParams` logic. By structuring the `contentData` array sequentially, I ensured that the `slideId` from the URL (e.g., `/2`) acts as a direct index key, instantly retrieving the correct "Welcome to the Wild" or "Concept Check" slide without complex lookup functions.

6. Comparison: Vanilla JS vs. React

Feature	Vanilla JS Approach	My React Implementation
UI Updates	Manual DOM manipulation (<code>document.querySelector</code>)	Declarative State (React updates DOM automatically)
Navigation	Separate HTML files for every page	<code>react-router-dom</code> swaps components instantly

Reusability	Copy-pasting HTML blocks	Importing Components (e.g., <Navbar />)
State	Variables in global scope	Hooks (useParams, localStorage inside useEffect)

References & Learning Resources

Throughout this refactoring process, I relied on the following documentation to guide my architectural decisions:

1. React Components & Fundamentals:
<https://react.dev/learn/your-first-component>
This is where I learned how to nest components and pass props.
2. React Router (SPA Routing):
<https://reactrouter.com/en/main/start/tutorial>
I used this to understand how to map URLs to specific components in App.jsx.
3. MDN Web Docs (React Getting Started):
https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started
Used for setting up the Node.js environment and understanding the build tools.
4. Understanding Hooks (useEffect, useState):
<https://react.dev/reference/react>
The official API reference I used to implement the useVideoAutoplay and LessonPlayer logic.
5. <https://www.youtube.com/watch?v=jO1lPbIUz4> - video on useeffect
6. <https://www.youtube.com/watch?v=L-1sP3Ljhsg> - video on useeffect
7. <https://www.youtube.com/watch?v=V9i3cGD-mts> - video on useState
8. <https://www.youtube.com/watch?v=dCLhUialKPQ> - This link was helpful in the building of the app overall.
9. <https://www.youtube.com/watch?v=UJbtnNsILjI> - video for useParams